FGV EMAp

Computação na nuvem - Thiago Araújo

DijkFood

Alex Júnio Maia de Oliveira

Bruno Ferreira Salvi

João Pedro Jerônimo de Oliveira

Thalis Ambrosim Falqueto

Rio de Janeiro

2026

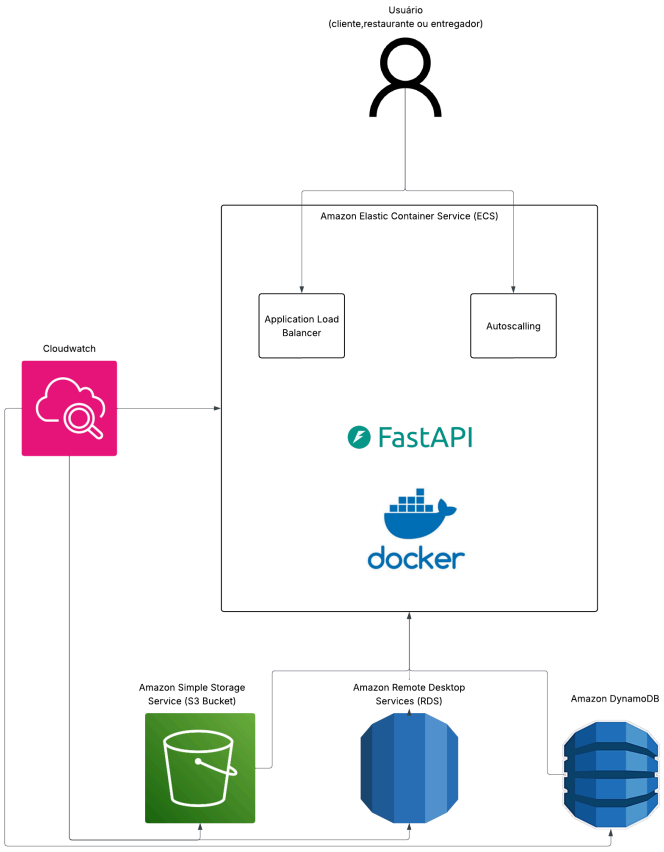
1 Introdução

O presente relatório detalha a arquitetura, as decisões de projeto e os resultados de implantação da plataforma DijkFood. O sistema foi desenvolvido para gerenciar o ciclo de vida de pedidos de delivery, calculando rotas no grafo viário de São Paulo através da biblioteca `osmnx` e hospedado de forma totalmente automatizada na nuvem AWS. O projeto foi implementado utilizando uma arquitetura de microsserviços, com foco em escalabilidade, resiliência e observabilidade, atendendo aos requisitos funcionais e não-funcionais estabelecidos.

2 Arquitetura

Em resumo, a arquitetura é composta por um Application Load Balancer (ALB) que distribui as requisições dos usuários para um cluster Amazon ECS (que contém os conteiners dos serviços) operando no modo Fargate, juntamente com Auto Scaling para gerenciar a capacidade de execução. A persistência de dados é realizada por um banco relacional Amazon RDS (PostgreSQL) para as operações transacionais, enquanto a telemetria de entregadores é armazenada em uma tabela NoSQL do Amazon DynamoDB. O grafo de São Paulo é pré-processado e armazenado no Amazon S3, permitindo inicialização rápida das instâncias ECS. Por fim, os logs e métricas são coletados utilizando o Amazon CloudWatch, garantindo monitoramento e observabilidade contínuos do serviço.

Tal arquitetura foi projetada para atender a requisitos funcionais rigorosos, como latência máxima de 500ms no percentil 95 (P95) sob carga de até 200 pedidos por segundo, e requisitos não-funcionais relacionados à escalabilidade, resiliência e segurança. Abaixo se encontra um diagrama de alto nível da arquitetura utilizada:



3 Fluxo de Dados

A arquitetura foi projetada para garantir separação de responsabilidades, alta disponibilidade e resiliência a picos de tráfego:

1. O **Application Load Balancer (ALB)** recebe o tráfego da Internet e distribui as requisições HTTP de forma balanceada.
2. O **Amazon ECS (Fargate)** hospeda os contêineres da aplicação (FastAPI), isolados em instâncias independentes.
3. **Rotas Transacionais (Pedidos, Cadastros):** A aplicação se comunica com um banco de dados relacional **Amazon RDS (PostgreSQL)** configurado em Multi-AZ para garantir a persistência atômica das entidades.
4. **Rotas de Telemetria (Rastreio 100ms):** A posição do entregador é desviada para uma tabela NoSQL no **Amazon DynamoDB**, evitando gargalos de I/O no banco relacional.
5. **Cache do Grafo:** Durante a inicialização, as instâncias ECS acessam o **Amazon S3** para realizar o download rápido do grafo `.graphml`, mitigando tempos de inicialização lentos.

4 Decisões de Projeto e Cumprimento de Requisitos

4.1 Modelagem de Dados e Telemetria (RDS vs. DynamoDB)

- **Requisito:** O projeto exige consistência transacional na criação de pedidos, mas suporta telemetria de entregadores a cada 100ms.

- **Alternativa Considerada:** Utilizar exclusivamente o RDS PostgreSQL para todas as operações.
- **Mudança de escolha:** Resolvemos trocar para uma persistência fazendo o uso de dois bancos (RDS + DynamoDB).
- **Justificativa:** Operações de rastreamento de posição exigem alta vazão de escrita. No cenário de evento especial (200 pedidos/segundo com atualizações a cada 100 ms), um banco relacional sofreria forte contenção de escritas ao mesmo tempo. O DynamoDB absorve essa carga isoladamente com latência previsível, impedindo que a saturação da telemetria degrade a leitura de dados transacionais pelos usuários e administradores, portanto a melhor escola seria integrar dois bancos de dados.

4.2 Computação Serverless vs. Instâncias Gerenciadas (ECS Fargate vs. EC2)

- **Requisito:** O serviço deve escalar horizontalmente e tolerar falhas de instâncias de computação.
- **Alternativa Considerada:** Provisionar instâncias EC2 gerenciadas via Auto Scaling Group.
- **Opção Escolhida:** Amazon ECS operando no modo AWS Fargate.
- **Justificativa:** O Fargate abstrai a camada do sistema operacional, eliminando a necessidade de gerenciamento de AMI e patching. A escalabilidade baseada em tarefas atende de forma mais ágil os picos de almoço/jantar. Além disso, integra-se perfeitamente às restrições do Learner Lab (uso exclusivo da `LabRole` de estudante).

4.3 Gestão de Inicialização e Cálculos de Rota

- **Requisito:** O algoritmo deve calcular as rotas utilizando caminhos mínimos sobre a rede real sem travar a API.
- **Alternativa Considerada:** O contêiner realizar o download em tempo real via API externa do OpenStreetMap.
- **Opção Escolhida:** Cache do grafo de um múnicipio de São Paulo (facilitando o cache) no Amazon S3 e implementação nativa do Algoritmo de Dijkstra.
- **Justificativa:** O download via OSMnx pode levar minutos, o que faria o Load Balancer identificar a nova instância ECS como defeituosa e encerrá-la. Ao criar um cache de backup no S3, o contêiner inicializa o grafo em disco em poucos segundos, permitindo que a API atenda os pedidos atômicos sem impacto de latência.

4.4 Segurança e Isolamento de Rede (Security Groups)

- **Requisito:** Os bancos de dados não devem estar expostos publicamente, e a comunicação deve ser restrita e controlada.
- **Decisão:** Implementação de Security Groups em cascata.
- **Justificativa:** O Application Load Balancer atua como o único ponto de entrada acessível via Internet. Os contêineres ECS aceitam tráfego exclusivamente originado pelo Security Group do ALB. Da mesma forma, a instância do RDS PostgreSQL foi configurada para não possuir IP público (`PubliclyAccessible=False`) e seu Security Group aceita conexões apenas do Security Group do ECS na porta 5432. Esse isolamento mitiga vetores de ataque externos direto à camada de dados.

5 Validação dos Requisitos Funcionais

A API REST implementada atende estritamente às regras de negócio mapeadas:

- **Cadastros e Validações:** Restrições de chaves estrangeiras, `CheckConstraints` (preços não negativos) e o controle de bloqueio de entregadores ocupados foram implementados.
- **Máquina de Estados de Entrega:** O fluxo `CONFIRMED -> PREPARING -> READY_FOR_PICKUP -> PICKED_UP -> IN_TRANSIT -> DELIVERED` possui transições blindadas em código, retornando erro `409 Conflict` para saltos inválidos.
- **Atribuição de Entregador:** A alocação ocorre utilizando coordenadas geográficas mapeadas em arestas do OSMnx para determinar a distância real do entregador em relação ao restaurante.

6 Resultados Experimentais de Carga

Os testes foram executados utilizando um simulador assíncrono desenvolvido com a biblioteca `aiohttp`, orquestrando requisições simultâneas para validação do limite de 500ms no percentil 95 (P95). O **seed** do banco de dados e a geração de tráfego foram executados logo após o provisionamento sem intervenção manual.

Segue uma imagem de uma das tentativas (essa, bem sucedida) demonstrando o comportamento do sistema durante os testes com 10 pedidos por segundo respectivamente:

```
Resultados para 10 RPS:
Total de Requisições: 1920
Latência Média: 219.57 ms
Latência P95: 515.54 ms
Pedidos agendados: 100
Pedidos completos: 100
Throughput efetivo de pedidos: 3.66 pedidos/s
AVISO: P95 acima de 500ms. ECS pode estar precisando de mais containers.
```

Abaixo, outro print do funcionamento de parte do simulador de carga: Disparo de um alarme em um serviço do ECS, durante teste de carga, alcançando uso de memória acima de 60%.

TargetTracking-service/dijkfood-v2-cluster/dijkfood-v2-service-AlarmLow-ec2de62d-e9a8-4b75-bd08-8b1e38942b1d	Em alarme	Ações habilitadas	-	2026-04-21 21:48:35	MemoryUtilization < 63 para 15 pontos de dados em 15 minutos

7 Desafios Encontrados e Soluções

Durante a fase de implantação e testes de estresse, dois desafios arquiteturais se destacaram, exigindo adaptações na infraestrutura como código (IaC):

1. **Drenagem de Conexões no ECS (Connection Draining):** No script de destruição automática, ao tentar excluir o cluster ECS imediatamente após o teste de 200 RPS, a AWS retornava o erro `ClusterContainsTasksException`. Compreendemos que o Load Balancer aguarda o processamento das últimas requisições na fila antes de encerrar as tarefas. A solução foi implementar um tempo de espera (`time.sleep`)

adequado no script, permitindo o desligamento gracioso (**graceful shutdown**) das instâncias.

2. **Alinhamento de Chaves no DynamoDB:** Houve um desacoplamento entre o design da aplicação (`courier_id` como Partition Key numérico) e o script de criação da tabela. A divergência causava falhas no SDK Boto3 ao injetar eventos em alta frequência. A infraestrutura foi refatorada para refletir exatamente os tipos (`N` e `S`) esperados pela API, além de aplicarmos o cast (`Decimal`) para compatibilizar os tipos `float` do Python com a exigência estrita do DynamoDB.

8 Modelo de Custos (Região us-east-1)

A tabela abaixo projeta os custos mensais estimados (em USD) operando ininterruptamente, demonstrando a elasticidade de custos entre o dimensionamento para a operação base e o auto-scaling em picos estressantes.

Recurso AWS	Operação Normal (Mensal)	Evento Especial (Mensal)
Application Load Balancer (1 segundo por conexão e e regra por solicitação)	$ **28.11** (1 TB/mês, 50 novas conexões e 50 solicitações por segundo)	$ **221.23** (25 TB/mês, 200 novas conexões e 200 solicitações por segundo)
ECS Fargate (2 vCPU, 4GB)	$ **144.16** (2 Tasks)	$ **576.65** (8 Tasks)
RDS PostgreSQL (db.t3.micro Multi-AZ, on-demand, com Proxy e com 20GB de armazenamento e de backup)	$ **54.68**	$ **54.68**
DynamoDB (Standard com 200 bytes por item)	$ **90.59** (1GB de armazenamento e 50 gravações e 50 leituras por segundo)	$ **337.59** (3,5GB de armazenamento e 200 gravações e 200 leituras por segundo)
Amazon S3 (Standard e com 20GB de armazenamento)	$ **0.23**	$ **0.23**
Custo Total Estimado	**$ 317.77**	**$ 1190.38**

Nota de Resiliência: O valor constante no RDS se deve ao provisionamento fixo com `MultiAZ=True`. A alta disponibilidade sacrifica a redução de custo base para garantir sobrevivência à queda de zonas, conforme o edital do projeto.

9 Considerações Finais

Ao longo do desenvolvimento e da validação deste projeto, foram realizadas diversas iterações para testar a elasticidade da arquitetura sob estresse. Avaliamos diferentes parâmetros processamento vertical para as tasks do ECS (variando de 256 de CPU / 512MB de memória até 1024 de CPU / 2048MB de memória) e alteramos também a tolerância do Target Group no Application Load Balancer. Ajustamos os parâmetros de Health Check (IntervalSeconds, TimeoutSeconds, Healthy e UnhealthyThresholdCount) para evitar o corte cedo demais de conexões durante o processamento do Algoritmo de Dijkstra. Por fim, prolongamos a duração das requisições no simulador de carga para tentar forçar a resposta da infraestrutura.

Apesar de todas as adequações estruturais, o Auto Scaling não chegou a ejetar corretamente novas instâncias durante os testes. Achamos que esse comportamento não vem de um erro de configuração, mas sim da mecânica intrínseca de monitoramento da nuvem. O escalonamento horizontal é mediado pelo Amazon CloudWatch, que coleta e agrega métricas de CPU em tempo discreto (geralmente de 1 minuto). Para evitar o provisionamento desnecessário gerado por ruídos de rede, as políticas de Target Tracking exigem que o limite de estresse seja ultrapassado de forma sustentada por vários ciclos de agregação (tipicamente 3 minutos contínuos) antes de disparar o alarme.

Portanto, testes de estresse rápidos, como os que fizemos — concentrados em segundos ou poucos minutos — são interpretados pela AWS como anomalias efêmeras, não satisfazendo o tempo de maturação do alarme. Fica a lição arquitetural de que o Auto Scaling é uma ferramenta desenhada para o crescimento orgânico e gradual de tráfego (ao longo de horas ou dias de operação). Por interpretação errada do que fazer, acabou não funcionando como esperado.

Para cenários que exigem absorção de picos instantâneos e massivos de requisições, a melhor prática não é o escalonamento reativo, mas sim a adoção de estratégias preventivas de Pre-warming, provisionando antecipadamente as tasks necessárias para suportar a carga inicial sem degradação do serviço.